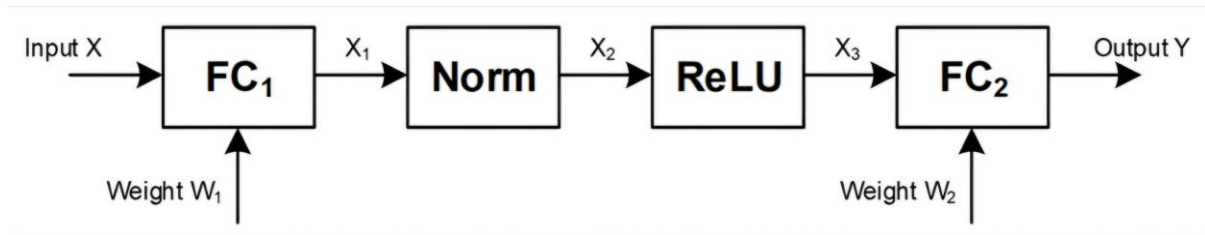


1. Project Overview

이번 final project는 Tiny Neural Network Accelerator를 구현하고, 이에 대응하는 testbench를 작성하여 기능을 검증하는 것이다. 구현 대상 신경망은 두개의 fully connected layer, normalization layer, ReLU activation layer로 구성된다. 두 fully connected layer에는 bias term이 없으며, 전체 연산 흐름은 다음과 같다.



따라서 본 설계가 최종적으로 수행하는 연산은 다음과 같이 표현할 수 있다.

1. $X1 = X * W1T$
2. $X2 = X1 / 32$
3. $X3 = \text{ReLU}(X2)$
4. $Y = X3 * W2T$

본 설계의 핵심은 이전 assn4에서 구현한 4x4 tiled matrix multiplication 기반 systolic array 구조를 재사용하여 FC1과 FC2의 matrix multiplication을 수행하는 것이다. 이를 완성하기 위해 이전 과제에서 만들었던 4x4 tiled matrix multiplication, weight-stationary dataflow, 8-bit CLA-based MAC 등을 그대로 사용할 것이다. 이를 만족하기 위해 본 설계에선 project module 내부에 기존 8*8 matrix multiplication engine인 top U_matmul을 하나만 사용하였다. top 내부에는 ctrl, sa8_4x4, cla16이 포함되어 있으며, sa8_4x4는 실제 4x4 systolic array 연산을 수행한다. 코드상으로도 top module 안에서 sa8_4x4 U_sa와 cla16 U_CLA16이 연결되어 있음을 확인할 수 있다.

본 accelerator는 이 하나의 matrix multiplication engine을 시간적으로 재사용한다. 먼저 FC1 단계에서는 X와 W1T를 입력으로 사용하여 X1을 계산한다. 이후 X1은 내부 buffer에 저장되고, normalization과 ReLU를 거쳐 X3로 변환된다. 마지막으로 FC2 단계에서는 동일한 matrix multiplication engine을 다시 사용하여 X3와 W2T를 곱하고 최종 output Y를 생성한다.

본 설계는 base task와 extra task를 모두 지원한다. Base task에서는 input X와 output Y가 8x8 matrix이며, extra task에서는 input X와 output Y가 16x8 matrix이다. README에서도 base dataset은 8-batch input/output이고, extra credit용 dataset은 16-batch input/output이라고 설명한다. Extra task의 경우, 별도의 추가 systolic array를 만들지 않고 16x8 input을 두 개의 8x8 block으로 나누어 처리한다. 즉, batch_mode = 0이면 base

8-batch를 처리하고, batch_mode = 1이면 16-batch를 상위 8개 row와 하위 8개 row로 나누어 동일한 hardware를 두 번 사용한다.

2. Target Network and Functionality

2-1. FC1 Functionality

FC1은 입력 matrix X와 첫 번째 transposed weight matrix W1T를 곱하여 intermediate output X1을 생성한다.

$$\rightarrow X1 = X * W1T$$

Base task에서 X는 8×8 matrix이고 W1T는 8×8 matrix이므로, FC1 결과 X1도 8×8 matrix이다.

$$\rightarrow X = 8 * 8, W1T = 8 * 8, X1 = 8 * 8$$

Extra task에서는 X가 16×8 matrix이고 W1T는 여전히 8×8 matrix이므로, FC1 결과 X1은 16×8 matrix가 된다.

$$\rightarrow X = 16 * 8, W1T = 8 * 8, X1 = 16 * 8$$

이때 본 설계의 matrix multiplication engine은 8*8 단위 연산을 수행하도록 구성되어 있으므로, extra task에서는 16*8 input을 두 개의 8*8 block으로 나누어 처리한다.

2-2. Norm Layer Functionality

Norm layer는 X1의 각 element를 32로 나누어 X2를 생성한다.

$$\rightarrow X2 = X1 / 32$$

본 설계에서는 이를 다음과 같은 방식으로 구현하였다.

$$\rightarrow \text{norm_shift16} = \text{norm_src} \ggg 5$$

$$\text{norm_x2} = \text{norm_shift16}[7:0]$$

여기서 (>>>5)는 signed arithmetic right shift이며, 32로 나누는 것과 같은 효과를 낸다. 단순 logical shift가 아니라 arithmetic shift를 사용해야 음수에 대해 부호가 유지된다. 이후 하위 8-bit를 취해 x2로 사용한다.

2-3. ReLU Layer Functionality

ReLU layer는 normalization 결과인 8-bit X2를 입력으로 받아 8-bit X3를 생성한다. ReLU의 정의는 다음과 같다.

-> **ReLU(x) = max(0, x)**

따라서 입력이 음수이면 0을 출력하고, 입력이 0 이상이면 입력값을 그대로 출력한다. 본 설계에서는 8-bit two's complement에서 MSB가 1이면 음수라는 점을 이용하여 ReLU를 구현하였다.

-> **norm_relu = (norm_x2[7]) ? 8'd0 : norm_x2**

즉 norm_x2[7] = 1이면 음수이므로 0을 출력하고, 그렇지 않으면 norm_x2를 그대로 X3로 저장한다. 본 설계에서는 X1_buf의 값을 element 단위로 읽어 Norm / ReLU를 수행한 뒤, 결과를 X3 memory에 저장하는 방식을 사용하였다.

2-4. FC2 Functionality

FC2는 ReLU 결과인 X3와 두 번째 transposed weight matrix W2T를 곱하여 최종 output Y를 생성한다. FC2 역시 FC1과 동일하게 bias term은 없으며, $X \cdot W^T = Y$ 방식으로 계산한다.

-> **$Y = X3 \times W2T$**

Base task에서는 다음과 같은 크기의 matrix multiplication이 수행된다.

-> **$X3 = 8 \times 8, W2T = 8 \times 8, Y = 8 \times 8$**

Extra task에서는 다음과 같다.

-> **$X3 = 16 \times 8, W2T = 8 \times 8, Y = 16 \times 8$**

2-5. Base and Extra Functionality

본 설계는 base와 extra를 모두 지원한다. Testbench의 EXTRA_TEST parameter와 accelerator의 batch_mode input을 사용하여 처리할 batch size를 선택한다. Testbench에서는 EXTRA_TEST = 0일 때 base dataset을 사용하고, EXTRA_TEST = 1일 때 extra dataset을 사용한다. 코드에서도 EXTRA_TEST 값에 따라 NUM_OUT을 64 또는 128로 설정하고, base 또는 extra input/output file을 선택한다.

-> **parameter EXTRA_TEST = 1**

localparam integer NUM_OUT = (EXTRA_TEST) ? 128 : 64

Base mode에서는 다음과 같이 동작한다.

-> **batch_mode = 0**

X size = 8×8

Y size = 8×8

NUM_OUT = 64

Extra mode에서는 다음과 같이 동작한다.

-> batch_mode = 1

X size = 16×8

Y size = 16×8

NUM_OUT = 128

2-6. Testbench Functionality Boundary

본 프로젝트에서 testbench는 functional logic을 수행하지 않고 testbench는 data read/write, top module 실행, state monitoring, output checking만 수행한다.

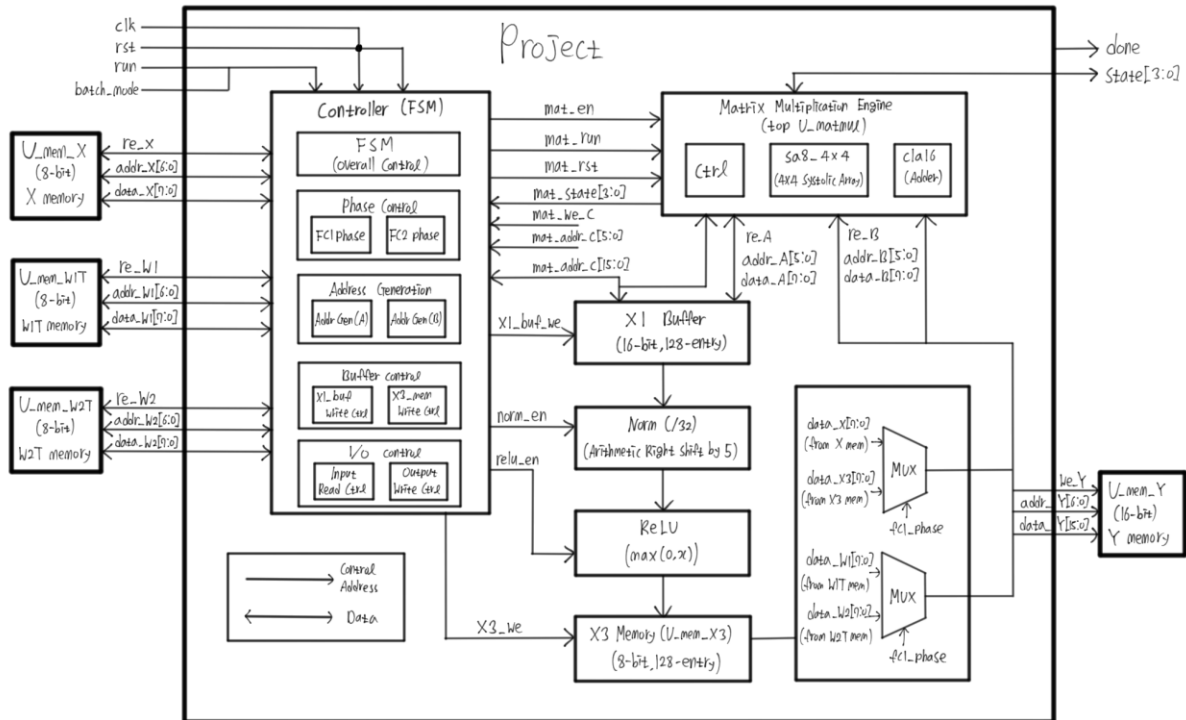
- 1. X, W1T, W2T memory initialization**
- 2. project module reset and run**
- 3. done signal 대기**
- 4. output memory Y read**
- 5. golden_Y와 비교**

3. Block diagram

3-1. Purpose of this section

1. X, W1T, W2T는 testbench memory에서 읽어온다.
2. FC1은 기존 Assignment 4 기반 8×8 matrix multiplication engine으로 수행한다.
3. FC1 결과 X1은 project 내부 X1_buf에 저장된다.
4. X1은 project 내부에서 Norm과 ReLU를 거쳐 X3로 변환된다.
5. FC2는 FC1과 동일한 matrix multiplication engine을 다시 사용한다.
6. 최종 Y는 output memory에 저장된다.

3-2. block diagram



3-3. How the architecture maps to the code

본 설계에서 전체 accelerator의 최상위 module은 project이다. project는 외부 testbench memory와 연결되는 port를 가진다.

project 내부에서는 FC1과 FC2의 동작 구간을 구분하기 위해 fc1_phase, fc2_phase 신호를 사용한다. FC1 phase에서는 matrix multiplication engine의 A 입력으로 data_X를 사용하고, B 입력으로 data_W1을 사용한다. FC2 phase에서는 A 입력으로 ReLU 결과인 x3_dout을 사용하고, B 입력으로 data_W2를 사용한다. 코드에서는 이 선택이 mux 형태로 구현되어있다.

-> assign mat_data_A = (fc1_phase) ? data_X : x3_dout

assign mat_data_B = (fc1_phase) ? data_W1 : data_W2

FC1 결과는 mat_data_C로 나오며, fc1_phase에서 mat_we_C가 올라오면 X1_buf에 저장된다. 이후 X1_buf의 값을 읽어 arithmetic right shift와 ReLU를 수행한 뒤 U_mem_X3에 저장한다. 코드에서는 Norm / ReLU가 다음과 같이 구현되어있다.

-> assign norm_src = X1_buf[norm_index]

assign norm_shift16 = norm_src >>> 5

assign norm_x2 = norm_shift16[7:0]

assign norm_relu = (norm_x2[7]) ? 8'd0 : norm_x2

FC2 결과는 최종 output Y이므로, fc2_phase에서 mat_we_C가 올라오면 we_Y가 올라가고, addr_Y, data_Y를 통해 output memory에 저장된다.

```
-> assign we_Y    = (fc2_phase) ? mat_we_C : 1'b0;
    assign addr_Y = mat_global_addr_C;
    assign data_Y = mat_data_C;
```

3-4. Base and extra architecture

Base task에서는 input X와 output Y가 8×8이다. 따라서 project는 block_idx = 0에 해당하는 8×8 block 하나만 처리한다.

```
-> X[0:7, :] → FC1 → Norm/ReLU → FC2 → Y[0:7, :]
```

Extra task에서는 input X와 output Y가 16×8이다. 그러나 16×8 전용 hardware를 따로 만들지 않고, 16×8 matrix를 두 개의 8×8 block으로 나누어 동일한 matrix multiplication engine을 두 번 사용한다.

```
-> Block 0: X[0:7, :] → Y[0:7, :]
```

```
Block 1: X[8:15, :] → Y[8:15, :]
```

코드에서는 block_idx를 사용하여 주소의 상위 bit를 결정한다.

```
-> assign mat_global_addr_A = {block_idx, mat_addr_A}
```

```
assign mat_global_addr_C = {block_idx, mat_addr_C}
```

따라서 block_idx = 0일 때는 address 0~63을 처리하고, block_idx = 1일 때는 address 64~127을 처리한다. 이 방식으로 동일한 hardware를 사용하면서 base와 extra를 모두 지원한다.

4. Module Structure

4-1. cla4, cla8, cla16 module

cla4, cla8, cla16은 carry lookahead adder 계층이다.

cla4: 4-bit carry lookahead adder

cla8: 두개의 cla4 modules로 구성된 8-bit adder

cla16: 두개의 cla8 modules로 구성된 16-bit adder

mad8 내부에서는 partial product를 누산하기 위해 cla16을 사용하고, top 내부에서는 partial result를 더하기 위해 cla16_U_CLA16을 사용한다. Top 내부에 cla16_U_CLA16이 존재하고, acc_data_P, acc_data_C, acc_out_C가 연결되어있다.

4-2. mad 8 module

mad8 module은 8-bit multiply-add 연산을 수행하는 module이다.

$$\rightarrow P = A \times B + C$$

mad8은 8-bit multiplication을 4-bit 단위 partial multiplication으로 나누어 수행한다. 이를 위해 다음 multiplier module들을 사용한다.

$\rightarrow$ **unsign_mult4, sign_mult4, mix_mult4**

그리고 partial result를 cla16을 이용해 누산한다.

4-3. se8 module

se8 module은 systolic array의 processing element이다.

1. weight register에 B 값을 저장한다.
2. 왼쪽에서 들어온 A 값을 오른쪽으로 전달한다.
3. 위에서 들어온 partial sum 또는 weight 값을 아래로 전달한다.
4. 내부 mad8을 이용하여 multiply-add를 수행한다.
5. busy와 enable pipeline을 통해 결과 valid timing을 조절한다.

Se8 내부에는 weight register가 있으며, pre_fill동안 data_from_top의 하위 8-bit를 weight로 저장한다. 이 구조가 weight-stationary dataflow에 해당한다.

$\rightarrow$ **weight stationary:** weight는 PE 내부에 저장되고, activation A와 partial sum이 이동하면서 계산된다.

4-4. sa8_4x4 module

Sa8_4x4 module은 실제 4x4 systolic array이다. 이 module은 4개의 8-bit A input과 4개의 8-bit B input을 받아 4개의 16-bit C output을 만든다.

1. weight-stationary 방식으로 B weight를 PE 내부에 prefill한다.
2. A data를 row 방향으로 전달한다.
3. partial sum을 column 방향으로 전달한다.
4. 4x4 tile의 matrix multiplication 결과를 출력한다.

4-5. ctrl module

Ctrl module은 8x8 matrix multiplication engine의 controller이다. Ctrl은 8x8 matrix multiplication을 직접 한번에 수행하지 않고, 4x4 tile 단위로 나누어 처리한다.

* Ctrl의 State

1. IDLE: run 신호를 기다리는 대기 상태
2. WHT_LOAD: B tile, weight tile을 memory에서 읽고 systolic array에 prefill
3. ACT_LOAD: A tile, activation tile을 memory에서 읽음
4. COMP: sa8_4x4에 A row를 입력하고 systolic array 연산 수행
5. ACCUM: partial sum을 더함
6. WRITE: 최종 4x4 tile result를 C memory로 write

* Ctrl의 buffer

1. buf_A: A tile 저장
2. buf_B: B tile 저장
3. buf_C: second partial result 저장
4. buf_P0: first partial result 저장
5. buf_P1: additional partial sum buffer

4-6. top module

Top module은 기존 assn4에서 만든 8x8 matrix multiplication engine이다. 본 설계에서는 project module 내부에서 이 top module을 U_matmul이라는 이름으로 한 번만 사용한다.

1. A memory와 B memory에서 8-bit 값을 읽는다.
2. 4x4 tile 단위로 matrix multiplication을 수행한다.
3. 내부 ctrl이 systolic array와 memory access를 제어한다.
4. K 방향 partial result를 accumulation한다.
5. 최종 8x8 C matrix를 16-bit output으로 write한다.

4-7. project module

Project module은 이번 설계에서의 최상위 module이다. Testbench는 project module을 instantiation하여 clk, rst, run, batch_mode를 입력하고, memory interface를 통해 X, W1T, W2T, Y memory와 연결한다. Testbench 코드에서도 project U_project가 instantiation되어 있으며, re_X, addr_X, data_X, re_W1, addr_W1, data_W1, re_W2, addr_W2, data_W2, we_Y, addr_Y, data_Y가 연결되어 있다.

1. 전체 network의 state를 제어한다.
2. FC1, Norm/ReLU, FC2의 순서를 제어한다.
3. base/extra mode를 batch_mode로 구분한다.
4. X/W1T/W2T memory read interface를 제어한다.
5. Y output memory write interface를 제어한다.
6. 기존 8×8 matrix multiplication engine인 top U_matmul을 재사용한다.
7. X1_buf와 U_mem_X3를 이용해 intermediate data를 저장한다.

이 FSM은 FC1을 먼저 실행하고, FC1 output을 X1_buf에 저장한 뒤, Norm/ReLU를 수행하여 U_mem_X3에 저장한다. 이후 동일한 top U_matmul을 다시 실행하여 FC2를 수행하고 최종 Y를 생성한다.

5. Base Task Dataflow

5-1. base input and output shape

Base task에서 input matrix X는 8×8 이고, 첫번째 weight matrix는 W1T이고, 크기는 8×8 이다. 따라서 FC1 결과 X1도 8×8 이다. ($X1 = X \times W1T = 8 \times 8$)

이후 Norm과 ReLU는 element-wise operation이므로 matrix shape은 변하지 않는다. ->

$$X2 = 8 \times 8, X3 = 8 \times 8$$

마지막 FC2에서는 X2와 W2T를 곱한다. 따라서 base task의 최종 output element 개수는 64개이다.

5-2. (1) Reading X and W1T

Base task가 시작되면 testbench는 input X와 weight W1T를 memory에 미리 넣어둔다. X는 U_mem_X, W1T는 U_mem_W1T에 저장되어 있다.

U_mem_X는 8-bit input memory이고, U_mem_W1T도 8-bit weight memory이다.

Testbench 코드에서 bitline = 8로 선언되어 있으므로, X와 W1T가 8-bit two's complement data로 들어간다는 것을 확인할 수 있다. project 내부에서는 FC1 phase일 때 matrix multiplication engine의 A 입력으로 data_X를 사용하고, B 입력으로 data_W1을 사용한다.

-> assign mat_data_A = (fc1_phase) ? data_X : x3_dout

assign mat_data_B = (fc1_phase) ? data_W1 : data_W2

5-2. (2) FC1 matrix multiplication

FC1에서 하는 연산 -> $X1 = X * W1T$

이 연산은 project 내부에서 직접 * 연산자로 수행되는 것이 아니라, 기존 Assignment 4에서 만든 8x8 matrix multiplication engine인 top U_matmul을 통해 수행된다. project module은 이 top U_matmul을 하나만 instantiation하고, FC1과 FC2에서 같은 engine을 재사용한다. Top U_matmul 내부에는 ctrl, sa8_4x4, cla16의 모듈이 있는데 이 중 sa8_4x4는 실제 4x4 systolic array이고, ctrl은 8x8 matrix multiplication을 4x4 tile 단위로 나누어 수행한다. top 내부에서 sa8_4x4 U_sa와 cla16 U_CLA16이 연결되어 있음을 코드에서 확인할 수 있다. Base 8x8 matrix multiplication은 4x4 tile 기준으로 나누어 처리되고, 각 output tile은 두개의 4-wide-tile을 누산하여 계산된다. Ctrl은 WHT_LOAD -> ACT_LOAD -> COMP -> ACCUM -> WRITE의 흐름으로 8x8 matrix multiplication을 수행한다.

5-2. (3) Storing FC1 output X1

FC1 결과는 mat_data_C로 나오고, mat_we_C가 올라오는 시점에 유효하다. Base task에서는 이 결과를 바로 최종 output memory에 쓰지 않고, project 내부의 X1_buf에 저장한다. FC1 phase에서 mat_we_C = 1이면, 현재 주소 mat_global_addr_C에 해당하는 X1_buf entry에 mat_data_C를 저장한다. X1_buf는 다음과 같이 16-bit, 128-entry buffer로 선언되어있고, Base task에서는 이 중 X1_buf[0]부터 X1_buf[63]까지만 사용된다.

5-2. (4) Norm layer

FC1이 끝나면 project는 S_NORM_RELU state로 이동한다. 이 state에서는 X1_buf에 저장된 16-bit FC1 output을 하나씩 읽어 Norm을 수행한다. Norm의 수식은 $X2 = X1/32$ 인데 이것은 코드에서 (>>>5)로 표현하였다.

5-2. (5) ReLU layer

Norm 결과 norm_x2는 8-bit signed value이다. ReLU는 $X3 = \max(0, X2)$ 의 수식을 따른다. 이 결과는 U_mem_X3에 저장되고, Base task에서는 U_mem_X3의 address 0~63에 저장된다.

5-2. (6) Reading X3 and W2T

Norm/ReLU가 끝나면 FC2가 시작된다. FC2에서는 A 입력으로 X3, B 입력으로 W2T를 사용한다. 이런 입력은 코드에서 mux를 통해 구현되었다. 즉 matrix multiplication engine은 X3와 W2T를 입력으로 받아 FC2를 수행한다.

5-2. (7) FC2 matrix multiplication and Y write

FC2는 $Y = X3 * W2T$ 의 연산을 수행한다. 이 연산도 FC1과 동일한 top U_matmul engine을 재사용하여 수행한다. FC2 결과는 최종 output이므로 project는 이를 Y output memory에 저장한다. 즉 FC2 phase에서 $mat_we_C = 1$ 이면, $we_Y = 1$ 이 되고, $addr_Y$ 주소에 $data_Y$ 값이 write된다. 최종적으로 testbench는 U_mem_Y에 저장된 64개의 output을 읽어 golden_Y와 비교한다.

5-3. Base task summary

1. U_mem_X에서 8×8 input X를 읽는다.
2. U_mem_W1T에서 8×8 weight W1T를 읽는다.
3. top U_matmul을 사용하여 FC1: $X \times W1T$ 를 수행한다.
4. FC1 결과 X1을 X1_buf에 저장한다.
5. X1을 arithmetic right shift by 5로 나누어 X2를 만든다.
6. X2에 ReLU를 적용하여 X3를 만든다.
7. X3를 U_mem_X3에 저장한다.
8. U_mem_W2T에서 W2T를 읽는다.
9. 같은 top U_matmul을 재사용하여 FC2: $X3 \times W2T$ 를 수행한다.
10. 최종 Y를 U_mem_Y에 저장한다.
11. testbench가 64개 output을 golden_Y와 비교한다.

6. Extra Task Dataflow and Test Method

6-1. Extra Task

Extra task는 16-batch input이 들어왔을 때 동일한 hardware가 어떻게 16×8 input을 처리하는지 설명한다. Base task와 extra task는 batch_mode input을 사용하여 구분한다.

* **batch_mode = 0** -> base 8-batch

* **batch_mode = 1** -> extra 16-batch

testbench에서는 EXTRA_TEST parameter를 사용하여 구분한다.

* **parameter EXTRA_TEST = 1**

6-2. Extra processing strategy

Extra task는 input = 16×8 matrix이고, weight matrix는 base task와 동일하게 8×8 matrix이다. Output element는 $16 \times 8 = 128$ 개가 나온다. 16×8 input을 별도 16×8 전용 hardware를 만들지 않고, 해결하는 방법은 16×8 input을 두 개의 8×8 block으로 나누어 순차적으로

처리하면 된다.

-> **block 0: rows 0~7 -> FC1 -> Norm/ReLU -> FC2 -> output [0:7,]**

block 1: rows 8~15 -> FC1 -> Norm/ReLU -> FC2 -> output [8:15,]

Extra_task에서 16*8 input을 두개의 8*8 block으로 나누기 위해 project module은 block_inx를 사용한다. block_inx = 0이면 상위 8개 row를 처리하고, block_idx = 1이면 하위 8개 row를 처리한다.

6-3. Extra dataflow

Extra task에서 FC1은 16*8 input X와 8*8 weight W1T를 곱해 16*8 X1을 만든다. 하지만 본 설계의 matrix multiplication engine은 8*8 단위로 동작하므로, 16*8 input을 두개의 8*8 block으로 나누어 순차적으로 처리한다. block_idx = 0일 때는 X 상위 8개 row를 처리하여 X1 상위 8개 row를 만들고, block_idx = 1일 때는 X 하위 8개 row를 처리하여 X 하위 8개 row를 만든다. FC1 결과는 X1_buf에 저장되며, address 0~63은 block 0, address 64~127은 block 1에 해당한다.

FC1이 끝난 뒤에는 해당 block의 X1_buf 값을 하나씩 읽어 Norm과 ReLU를 수행한다. Norm은 $X1 / 32$ 연산이며, 본 설계에서는 arithmetic right shift by 5로 구현하였다. 이후 ReLU는 8-bit signed value의 sign bit를 확인하여 음수이면 0을 출력하고, 양수이면 원래 값을 유지한다. 이 결과는 X3 memory에 저장된다. norm_index = {block_idx, norm_cnt [5:0]}로 구성되므로, block 0의 ReLU 결과는 X3[0:63], block 1의 ReLU 결과는 X3[64:127]에 저장된다.

FC2에서는 ReLU 결과인 X3와 두 번째 weight matrix W2T를 곱하여 최종 output Y를 만든다. FC2 역시 FC1과 동일한 matrix multiplication engine을 재사용한다. block_idx = 0일 때는 (X3의 상위 8개 row) × W2T를 수행하여 Y 상위 8개 row를 만들고, block_idx = 1일 때는 (X3의 하위 8개 row) × W2T를 수행하여 Y 하위 8개 row를 만든다. 최종 결과는 we_Y, addr_Y, data_Y를 통해 output memory에 write된다.

7. Testbench Structure

```

70     project U_project (
71         .clk      (clk),
72         .rst      (rst),
73         .run      (run),
74         .batch_mode (batch_mode),
75         .done     (done),
76         .state    (state),
77
78         .re_X     (re_X),
79         .addr_X   (addr_X),
80         .data_X   (data_X),
81
82         .re_W1    (re_W1),
83         .addr_W1  (addr_W1),
84         .data_W1  (data_W1),
85
86         .re_W2    (re_W2),
87         .addr_W2  (addr_W2),
88         .data_W2  (data_W2),
89
90         .we_Y     (we_Y),
91         .addr_Y   (addr_Y),
92         .data_Y   (data_Y)
93     );

```

Testbench는 project module을 다음과 같이 instantiation한다. 이 연결을 통해 testbench는 project의 memory interface와 control signals를 관찰하고, output을 검증한다. 그리고 위에서도 설명했듯이 EXTRA_TEST parameter를 사용하여 base와 extra를 선택한다. Base test를 수행하려면 0으로 설정하고, extra test를 수행하려면 1로 설정해야한다. 또한 Batch_mode는 EXTRA_TEST 값을 따라간다.

```

mem_behavior #(
    .firmware ("data/model/w1_t_hex.txt"),
    .bitline  (8),
    .bitaddr  (6),
    .binary   (0)
) U_mem_W1T (
    .clk  (clk),
    .en   (re_W1),
    .we   (1'b0),
    .addr (addr_W1),
    .din  (8'd0),
    .dout (data_W1)
);

mem_behavior #(
    .firmware ("data/inout_base/x_hex.txt"),
    .bitline  (8),
    .bitaddr  (7),
    .binary   (0)
) U_mem_X (
    .clk  (clk),
    .en   (re_X),
    .we   (1'b0),
    .addr (addr_X),
    .din  (8'd0),
    .dout (data_X)
);

mem_behavior #(
    .firmware ("data/model/w2_t_hex.txt"),
    .bitline  (8),
    .bitaddr  (6),
    .binary   (0)
) U_mem_W2T (
    .clk  (clk),
    .en   (re_W2),
    .we   (1'b0),
    .addr (addr_W2),
    .din  (8'd0),
    .dout (data_W2)
);

mem_behavior #(
    .firmware ("data/inout_extra/x_hex.txt"),
    .bitline  (8),
    .bitaddr  (7),
    .binary   (0)
) U_mem_X (
    .clk  (clk),
    .en   (re_X),
    .we   (1'b0),
    .addr (addr_X),
    .din  (8'd0),
    .dout (data_X)
);

```

Testbench는 mem_behavior module을 사용하여 input과 weight memory를 만든다. Base

test는 x_hex.txt, w1_t_hex.txt, w2_t_hex.txt을 사용하고 extra test에서는 input file만 extra 용으로 바꾼다. Weight file은 base와 extra가 동일하게 사용한다.

```
for (i = 0; i < NUM_OUT; i = i + 1) begin
    tb_addr_Y <= i[6:0];

    @(posedge clk);
    #2;

    if (mem_dout_Y == golden_Y[i]) begin
        num_correct = num_correct + 1;
        $display("[CORRECT] Y[%0d] = %h", i, mem_dout_Y);
    end else begin
        num_error = num_error + 1;
        $display("[WRONG] Y[%0d] = %h, expected = %h",
            i, mem_dout_Y, golden_Y[i]);
    end
end
end
```

project가 모든 계산을 끝내면 done = 1이 된다. Testbench는 done을 기다린 뒤, output memory를 읽어 golden output과 비교한다. 이후 testbench는 tb_check = 1로 설정하고, output memory address를 0부터 NUM_OUT-1까지 순차적으로 읽는다. 여기서 NUM_OUT은 base에서는 64, extra에서는 128이다.

Output memory U_mem_Y는 project가 계산한 최종 결과를 저장하는 memory이다. 정답 output은 golden_Y 배열에 따로 저장된다. 즉 U_mem_Y는 계산 결과 저장용이고, golden_Y는 비교용 expected output이다. 계산 중에는 project가 addr_Y, we_Y, data_Y를 이용해 output memory에 값을 쓴다. 계산이 끝난 뒤에는 testbench가 tb_check = 1로 바꾸고, tb_addr_Y를 0부터 NUM_OUT-1까지 증가시키며 output memory를 읽는다. 읽은 값 mem_dout_Y와 정답 golden_Y[i]를 비교하여 correct/error count를 출력한다.

8. Testbench display 결과

8-1. base task

```
[CORRECT] Y[0] = ff24
[CORRECT] Y[1] = 003d
[CORRECT] Y[2] = 00cd
[CORRECT] Y[3] = 0023
[CORRECT] Y[4] = ff10
[CORRECT] Y[5] = ffc3
[CORRECT] Y[6] = 0039
[CORRECT] Y[7] = 0099
[CORRECT] Y[8] = fff0
[CORRECT] Y[9] = ffe0
[CORRECT] Y[10] = 0022
[CORRECT] Y[11] = 0012
[CORRECT] Y[12] = fff0
[CORRECT] Y[13] = 0022
[CORRECT] Y[14] = 001a
[CORRECT] Y[15] = 0010
[CORRECT] Y[16] = fef2
[CORRECT] Y[17] = 0033
[CORRECT] Y[18] = 00d7
[CORRECT] Y[19] = 0014
[CORRECT] Y[20] = fee6
[CORRECT] Y[21] = ff92
[CORRECT] Y[22] = 005a
[CORRECT] Y[23] = 00d4
[CORRECT] Y[24] = ffbb
[CORRECT] Y[25] = ff24
[CORRECT] Y[26] = 00d4
[CORRECT] Y[27] = ffd8
[CORRECT] Y[28] = ff0c
[CORRECT] Y[29] = 0085
[CORRECT] Y[30] = 007f
[CORRECT] Y[31] = 00f5
[CORRECT] Y[32] = 00fd
[CORRECT] Y[33] = 0031
[CORRECT] Y[34] = fff3
[CORRECT] Y[35] = ff58
[CORRECT] Y[36] = ffc5
[CORRECT] Y[37] = ff57
[CORRECT] Y[38] = 0046
[CORRECT] Y[39] = ff67
[CORRECT] Y[40] = ff80
[CORRECT] Y[41] = 0003
[CORRECT] Y[42] = ff13
[CORRECT] Y[43] = 006c
[CORRECT] Y[44] = 016a
[CORRECT] Y[45] = ffba
[CORRECT] Y[46] = 0018
[CORRECT] Y[47] = ff6f
[CORRECT] Y[48] = fe82
[CORRECT] Y[49] = 0120
[CORRECT] Y[50] = ffbf
[CORRECT] Y[51] = 00e3
[CORRECT] Y[52] = 0017
[CORRECT] Y[53] = fee7
[CORRECT] Y[54] = ff8a
[CORRECT] Y[55] = fffc
[CORRECT] Y[56] = fef4
[CORRECT] Y[57] = 00da
[CORRECT] Y[58] = ffa1
[CORRECT] Y[59] = 0101
[CORRECT] Y[60] = 00ee
[CORRECT] Y[61] = ff8d
[CORRECT] Y[62] = 0017
[CORRECT] Y[63] = ff5e
```

```
=====
RESULT: correct = 64, error = 0, total = 64
```

```
PASS: All outputs are correct.
=====
```

8-2. extra task

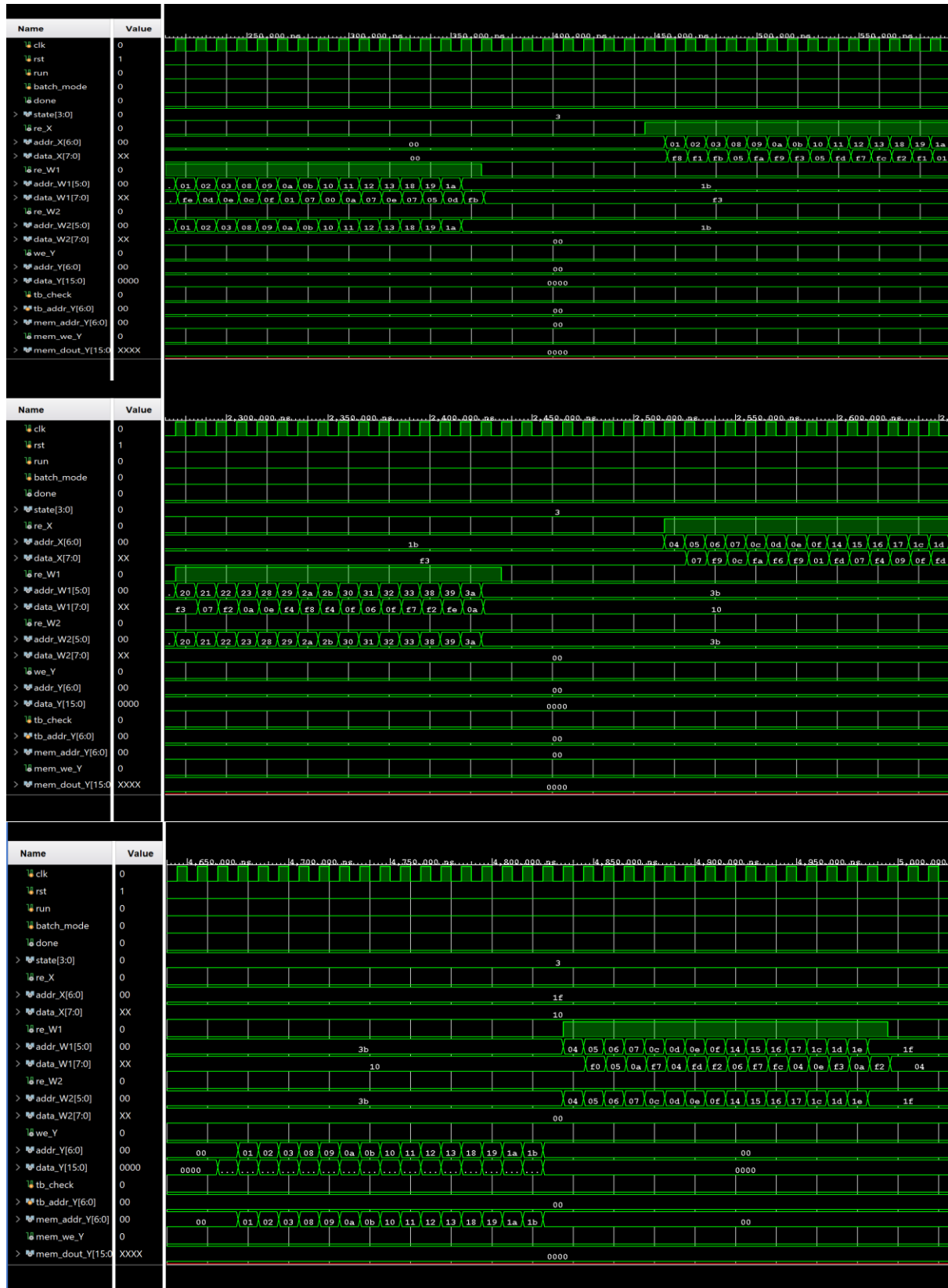
```
[CORRECT] Y[0] = fff7
[CORRECT] Y[1] = ff88
[CORRECT] Y[2] = 00b5
[CORRECT] Y[3] = ffeF
[CORRECT] Y[4] = ff65
[CORRECT] Y[5] = 0088
[CORRECT] Y[6] = 0099
[CORRECT] Y[7] = 004c
[CORRECT] Y[8] = ff28
[CORRECT] Y[9] = 0015
[CORRECT] Y[10] = 000c
[CORRECT] Y[11] = 002b
[CORRECT] Y[12] = feec
[CORRECT] Y[13] = ffb5
[CORRECT] Y[14] = ffb5
[CORRECT] Y[15] = 00bd
[CORRECT] Y[16] = 001c
[CORRECT] Y[17] = ff98
[CORRECT] Y[18] = ff89
[CORRECT] Y[19] = ff7e
[CORRECT] Y[20] = 005e
[CORRECT] Y[21] = ffa6
[CORRECT] Y[22] = 0028
[CORRECT] Y[23] = 002e
[CORRECT] Y[24] = ff74
[CORRECT] Y[25] = ffe9
[CORRECT] Y[26] = 0091
[CORRECT] Y[27] = ffff
[CORRECT] Y[28] = feec
[CORRECT] Y[29] = fff1
[CORRECT] Y[30] = 000d
[CORRECT] Y[31] = 00a6
[CORRECT] Y[32] = 005e
[CORRECT] Y[66] = fff7
[CORRECT] Y[67] = ffae
[CORRECT] Y[68] = fff6
[CORRECT] Y[69] = ffa2
[CORRECT] Y[70] = 002c
[CORRECT] Y[71] = ff70
[CORRECT] Y[72] = ff58
[CORRECT] Y[73] = 00b7
[CORRECT] Y[74] = 000b
[CORRECT] Y[75] = 0091
[CORRECT] Y[76] = 000b
[CORRECT] Y[77] = ffd1
[CORRECT] Y[78] = 0024
[CORRECT] Y[79] = ff79
[CORRECT] Y[80] = fe38
[CORRECT] Y[81] = 0192
[CORRECT] Y[82] = ffd1
[CORRECT] Y[83] = 0120
[CORRECT] Y[84] = 0077
[CORRECT] Y[85] = feac
[CORRECT] Y[86] = ffd1
[CORRECT] Y[87] = ffa0
[CORRECT] Y[88] = ffe9
[CORRECT] Y[89] = ffc3
[CORRECT] Y[90] = 0039
[CORRECT] Y[91] = ffc5
[CORRECT] Y[92] = feF5
[CORRECT] Y[93] = ffdE
[CORRECT] Y[94] = 0057
[CORRECT] Y[95] = 005e
[CORRECT] Y[96] = 003f
[CORRECT] Y[97] = 0060
[CORRECT] Y[98] = fff6
```

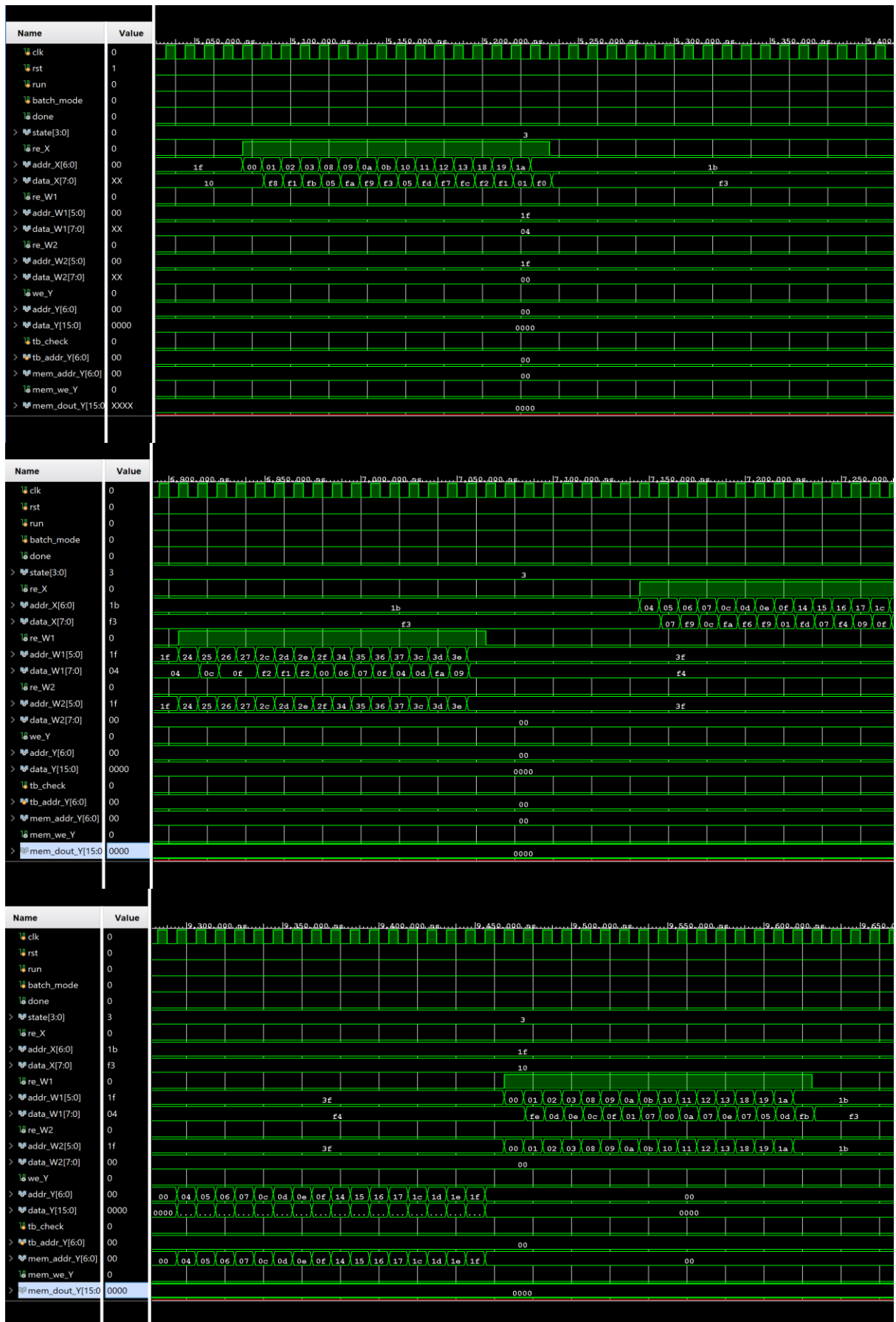
```
[CORRECT] Y[33] = 00c0
[CORRECT] Y[34] = ffd1
[CORRECT] Y[35] = ffb0
[CORRECT] Y[36] = 0004
[CORRECT] Y[37] = ff20
[CORRECT] Y[38] = 009e
[CORRECT] Y[39] = ff1e
[CORRECT] Y[40] = ff32
[CORRECT] Y[41] = 005f
[CORRECT] Y[42] = ffaF
[CORRECT] Y[43] = 009b
[CORRECT] Y[44] = ff46
[CORRECT] Y[45] = ffa7
[CORRECT] Y[46] = ff49
[CORRECT] Y[47] = 0016
[CORRECT] Y[48] = ffaa
[CORRECT] Y[49] = 0058
[CORRECT] Y[50] = ffe3
[CORRECT] Y[51] = 006b
[CORRECT] Y[52] = 005a
[CORRECT] Y[53] = ffd9
[CORRECT] Y[54] = ffe7
[CORRECT] Y[55] = ffb6
[CORRECT] Y[56] = ffe6
[CORRECT] Y[57] = ff8d
[CORRECT] Y[58] = 0065
[CORRECT] Y[59] = ffe0
[CORRECT] Y[60] = ffb2
[CORRECT] Y[61] = 0058
[CORRECT] Y[62] = 007e
[CORRECT] Y[63] = 004b
[CORRECT] Y[64] = 009c
[CORRECT] Y[65] = 0046
[CORRECT] Y[99] = ffd5
[CORRECT] Y[100] = 0012
[CORRECT] Y[101] = ffa4
[CORRECT] Y[102] = 005a
[CORRECT] Y[103] = ff77
[CORRECT] Y[104] = fffb
[CORRECT] Y[105] = 00d6
[CORRECT] Y[106] = 0057
[CORRECT] Y[107] = 0045
[CORRECT] Y[108] = 0029
[CORRECT] Y[109] = 0012
[CORRECT] Y[110] = 00bb
[CORRECT] Y[111] = feea
[CORRECT] Y[112] = 00be
[CORRECT] Y[113] = 0052
[CORRECT] Y[114] = ffc3
[CORRECT] Y[115] = ff7c
[CORRECT] Y[116] = ffe6
[CORRECT] Y[117] = ff44
[CORRECT] Y[118] = 0034
[CORRECT] Y[119] = ff6a
[CORRECT] Y[120] = ffe7
[CORRECT] Y[121] = ff37
[CORRECT] Y[122] = feb0
[CORRECT] Y[123] = ffa4
[CORRECT] Y[124] = 0118
[CORRECT] Y[125] = ffc7
[CORRECT] Y[126] = 0025
[CORRECT] Y[127] = fff2
```

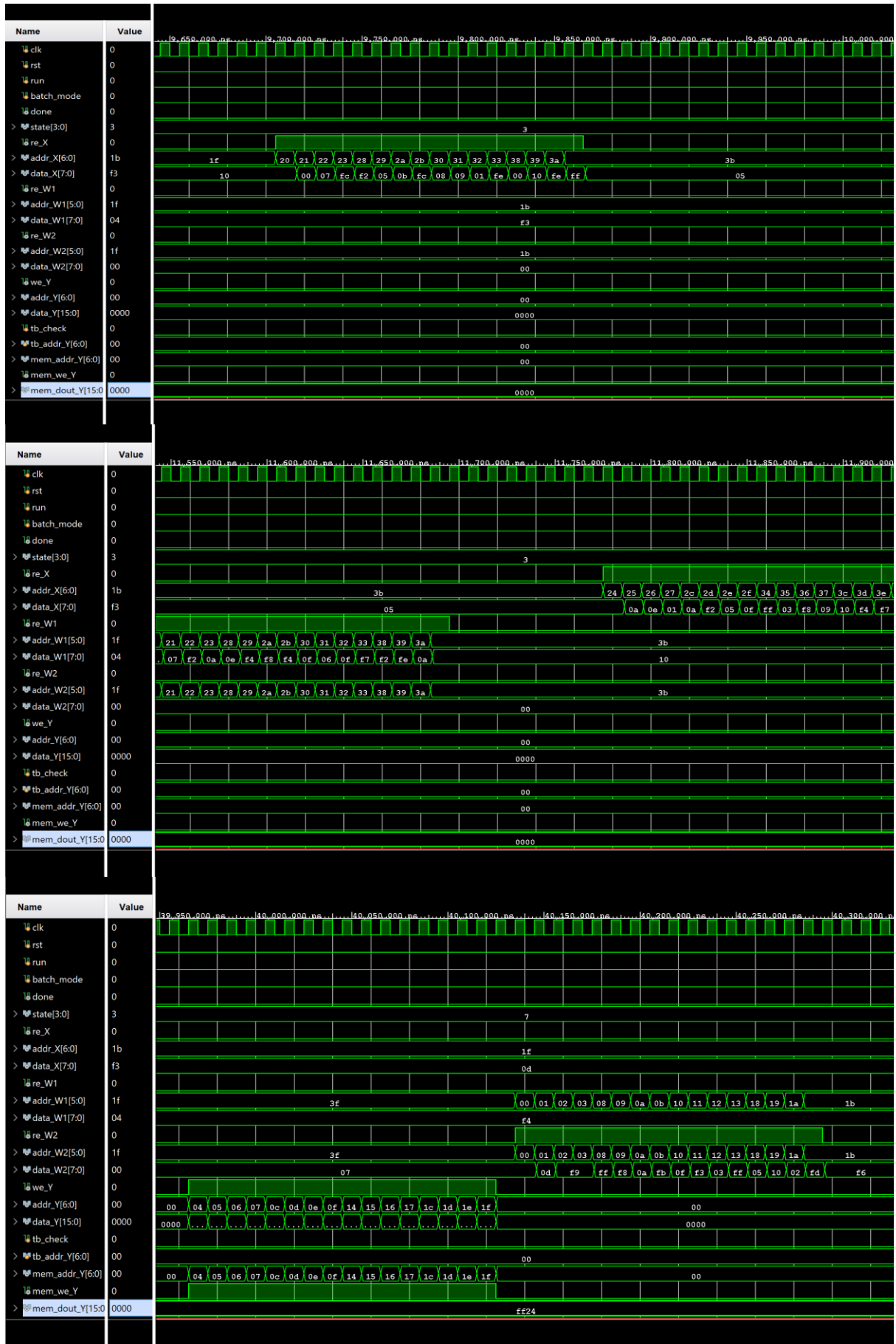
```
=====
RESULT: correct = 128, error = 0, total = 128
PASS: All outputs are correct.
```


9. Testbench waveform 결과

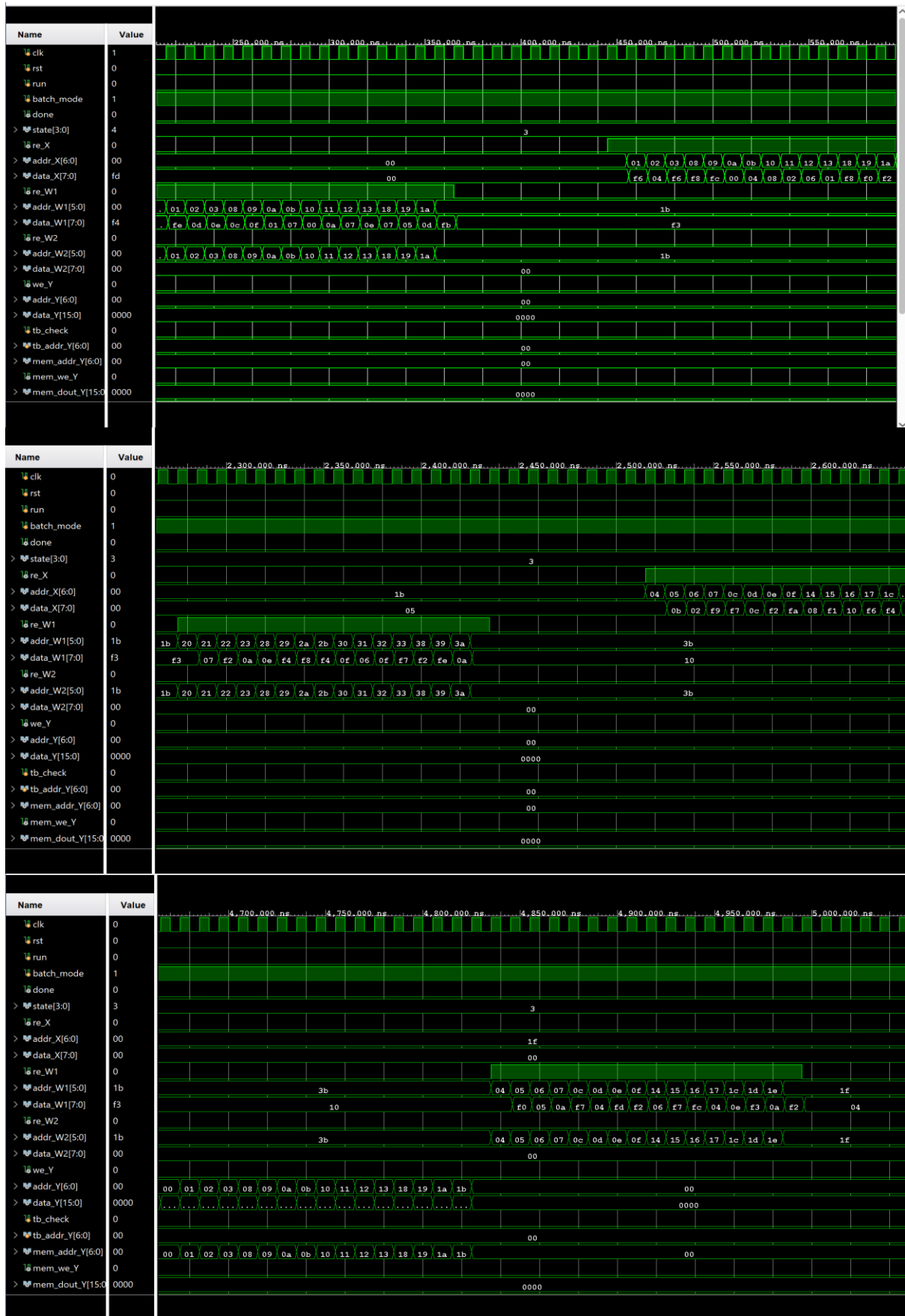
9-1. base task

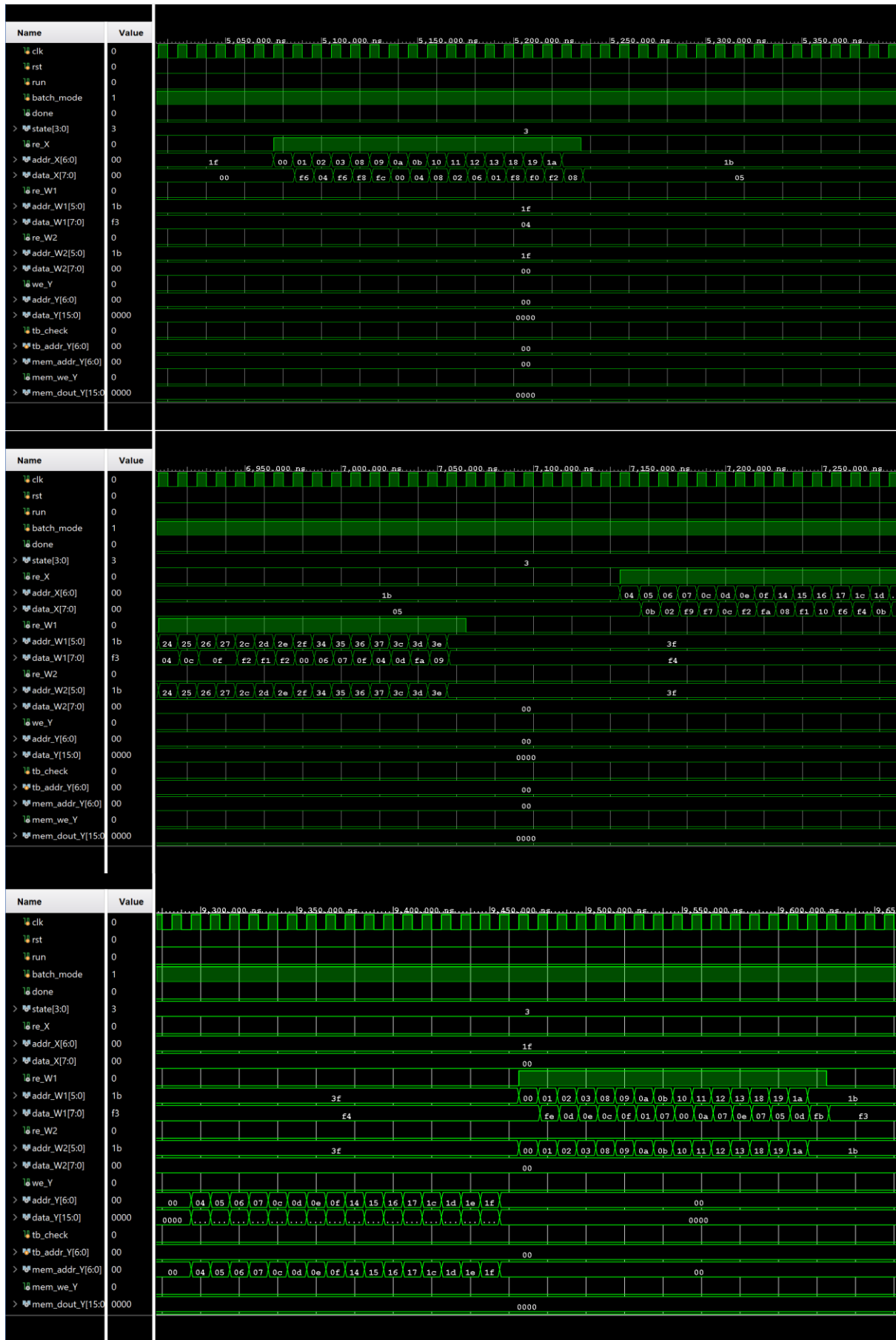


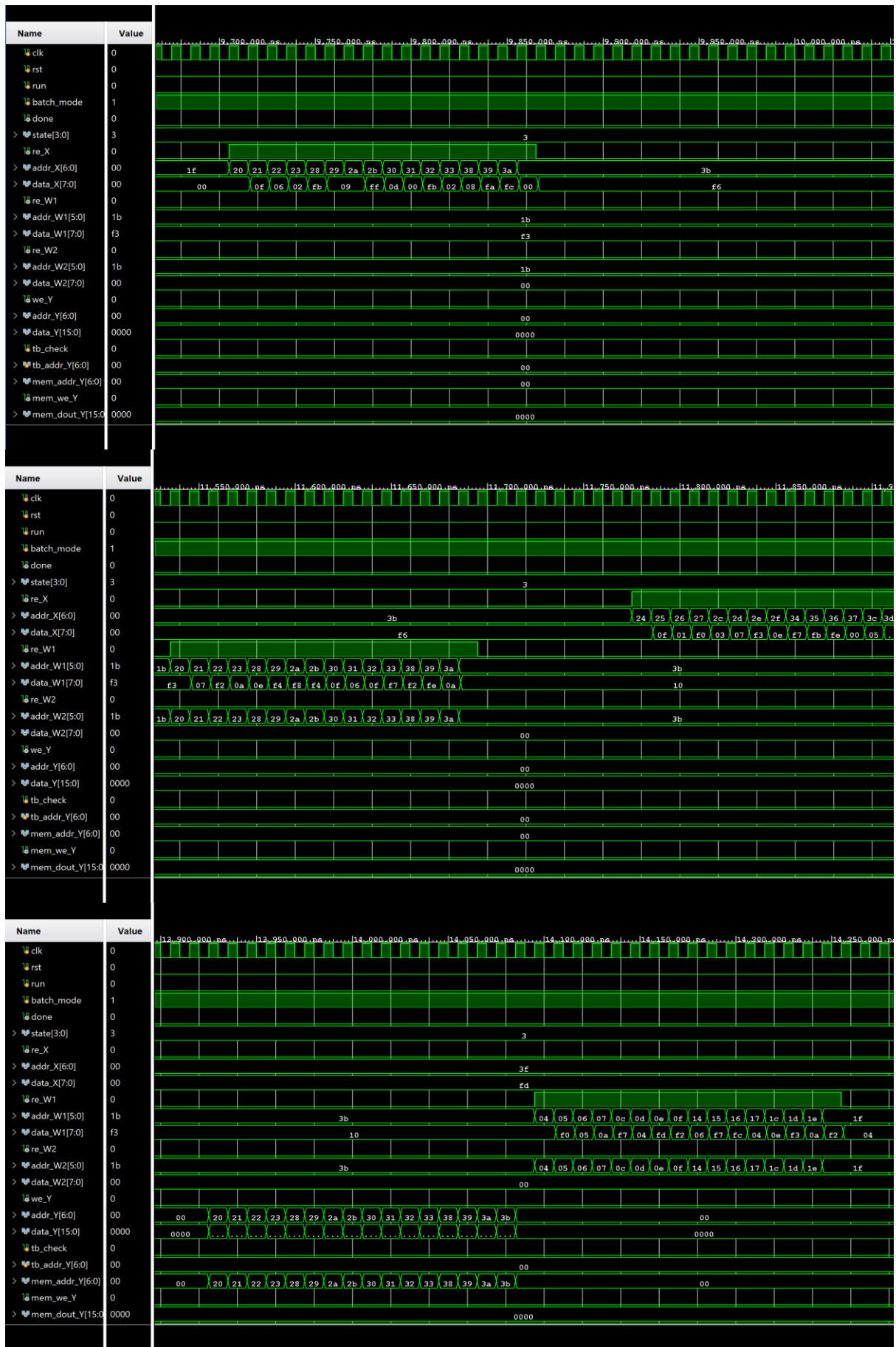


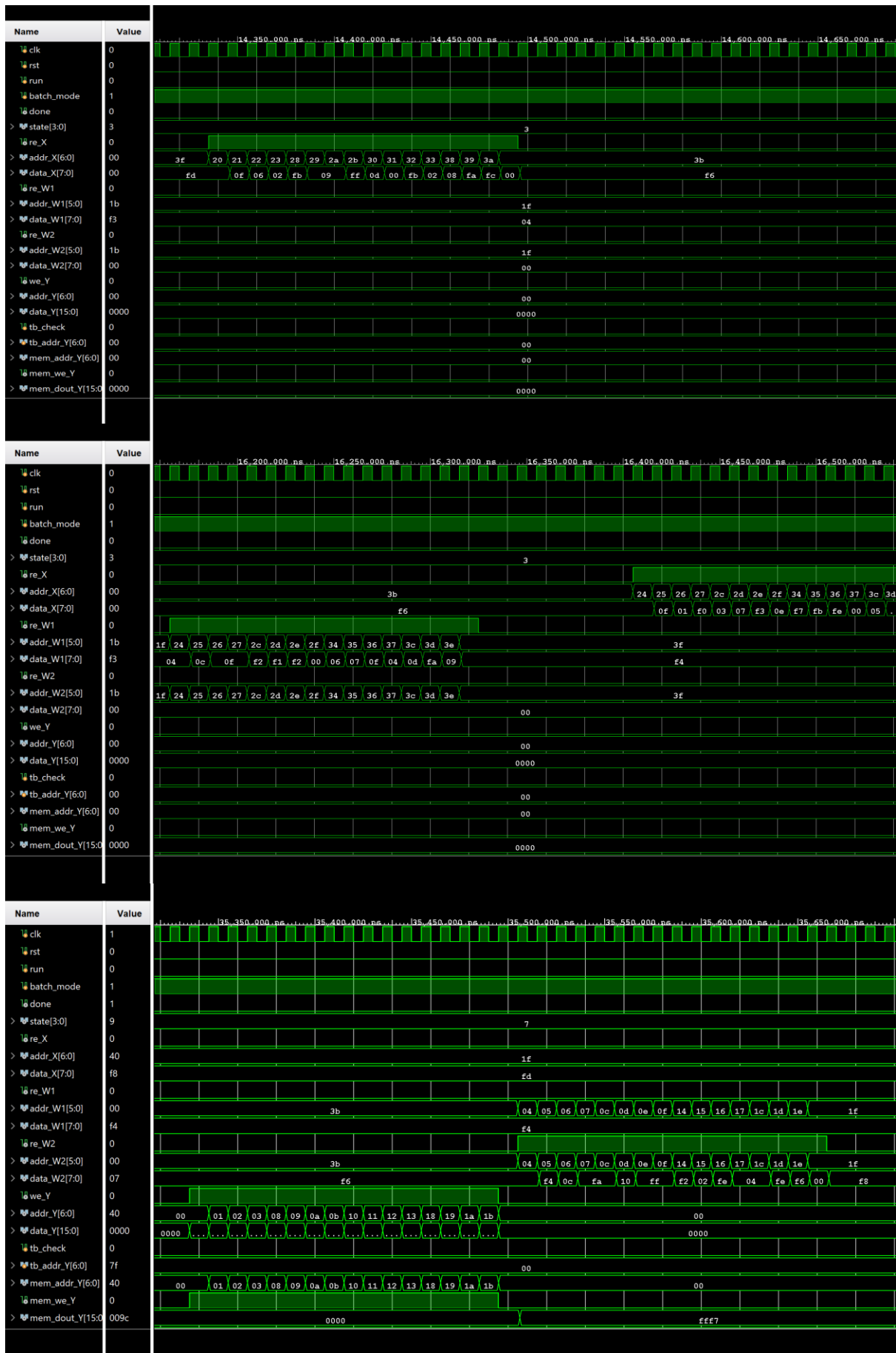


9-2. extra task









10. Total memory capacity [kB]

10-1. concept

Memory capacity : 데이터를 저장하기 위해 사용되는 총 저장공간

-> hardware 내부에 존재하는 buffer, memory, register array의 크기를 계산
계산식) memory capacity = entry 개수 x entry bit width & 1kB = 1024Byte

10-2. 내 hardware에서 어느 부분일까?

A. project 내부 buffer

project 내부에 FC1 결과를 저장하는 X1_buf가 있다. 이것은 FC1 결과 X1을 저장한다.

-> reg [15:0] X1_buf [0:127]

계산) $X1_buf = 128 \times 16bit = 2048 bit = 256B$

또 X3을 저장하는 내부 memory가 있는데 bitaddr = 7이므로 주소 개수는 $2^7 = 128$ entries, bitline = 8이므로 $X3 memory = 128 \times 8 bit = 1024 bit = 128B$ 이다.

B. matmul engine 내부 tile buffer

기존 top → ctrl 내부에는 4×4 tile 연산을 위한 buffer들이 있다.

-> buf_A [0:3][0:3] : 8-bit -> $buf_A = 4 \times 4 \times 8bit = 128bit = 16B$

buf_B [0:3][0:3] : 8-bit -> $buf_B = 4 \times 4 \times 8bit = 128bit = 16B$

buf_C [0:3][0:3] : 16-bit -> $buf_C = 4 \times 4 \times 16bit = 256bit = 32B$

buf_P0 [0:3][0:3] : 16-bit -> $buf_P0 = 4 \times 4 \times 16bit = 256bit = 32B$

buf_P1 [0:3][0:3] : 16-bit -> $buf_P1 = 4 \times 4 \times 16bit = 256bit = 32B$

합: ctrl tile buffers = $16 + 16 + 32 + 32 + 32 = 128B$

이 buffer들은 ctrl이 4×4 tile을 읽고, partial sum을 저장하고, 최종 4×4 결과를 쓰기 위해 사용됨.

C. systolic array 내부 weight register

sa8_4x4는 4×4 PE 구조라서 총 16개의 se8이 있고, 각 se8에는 8-bit weight register가 있음.

-> reg [7:0] weight

계산) $16PE \times 8bit = 128bit = 16B$

* total on-accelerator memory = $256+128+128+16 = 528B = 0.516kB$

추가로 testbench의 input/output memory까지 포함하면

-> X memory = $128 \times 8bit = 128B$

W1T memory = $64 \times 8\text{bit} = 64\text{B}$

W2T memory = $64 \times 8\text{bit} = 64\text{B}$

Y output memory = $128 \times 16\text{bit} = 256\text{B}$

External I/O memory = 512B = 0.50kB

따라서 testbench I/O memory까지 포함하면 $528\text{B} + 512\text{B} = 1040\text{B} = 1.02\text{kB}$

11. Peak bandwidth [MB/s]

11-1. concept

Bandwidth는 한 초에 memory interface를 통해 이동할 수 있는 데이터 양이다.

계산식) Bandwidth = data width per cycle x clock frequency

1Byte = 8bit & 100MHz = 100000000 cycles/s

11-2. 내 hardware에서 어느 부분일까?

내 project의 외부 memory interface

-> X memory: data_X[7:0]

W1 memory: data_W1[7:0]

W2 memory: data_W2[7:0]

Y memory: data_Y[15:0]

데이터 폭은 X read = 8bit, W read = 8bit, Y write = 16bit이다.

1. 현재 FSM이 실제로 사용되는 peak bandwidth

현재 FSM에서 한 cycle에 실제로 가장 많이 이동하는 것은 Y write 16bit이다.

Peak active transfer = $16\text{bit} / \text{cycle} = 2\text{B} / \text{cycle}$

Clock이 100MHz이므로, $2\text{B} / \text{cycle} \times 100\text{MHz} = 200\text{MB/s}$

-> **Actual peak bandwidth used by the implemented FSM $\approx 200\text{ MB/s}$**

2. 포트 폭 기준 aggregate bandwidth

만약 X/W/Y 포트를 모두 동시에 사용할 수 있다고 단순히 포트 폭만 합치면

$X\text{ read } 8\text{bit} + W\text{ read } 8\text{bit} + Y\text{ write } 16\text{bit} = 32\text{bit/cycle} = 4\text{B/cycle}$

-> $4\text{B/cycle} \times 100\text{MHz} = 400\text{MB/s}$

12. Peak performance [GOPS]

12-1. concept

Peak performance는 hardware가 이상적으로 낼 수 있는 최대 연산량이다.

보통 MAC 연산을 기준으로 계산함 -> **1MAC=multiplication + addition=2 operations**

12-2. 내 hardware에서 어느 부분일까?

연산은 4x4 systolic array로 진행한다. sa8_4x4는 4x4=16개의 se8을 생성하고, 각 se8 안에는 mad8이 있다. 그리고 mad8은 8-bit multiply-add를 수행한다.

-> **for (i = 0; i < 4; i = i + 1)**

for (j = 0; j < 4; j = j + 1)

se8 U_SE (...)

sa8_4x4 내부 generate문에서 4x4로 se8로 생성한다. 각 se8 내부에는 mad8 U_MAD8이 있고, PE 1개당 MAD 연산기 1개다.

1. Nominal systolic-array peak

만약 각 PE가 매 cycle 1 MAC을 수행할 수 있다고 이상적으로 보면

-> $16\text{PE} \times 1\text{MAC/cycle} \times 2\text{ ops/MAC} = 32\text{ops/cycle}$

100MHz에서 계산하면 $32\text{ops/cycle} \times 100\text{MHz} = \mathbf{3.2GOPS}$ 다.

2. Implemented MAD8 latency를 고려한 effective peak

코드 속 mad8은 내부적으로 count0~3을 돌면서 8-bit multiply-add를 여러 cycle에 걸쳐 수행하므로 완전한 1-cycle MAC이라고 보기힘들다.

보수적으로 1MAC을 약 4 cycle로 보면

-> $16\text{PE} \times 1\text{MAC} / 4\text{cycles} \times 2\text{ops/MAC} = 8\text{ops/cycle}$

100MHz에서 계산하면 $8\text{ops/cycle} \times 100\text{MHz} = \mathbf{0.8GOPS}$ 다.

13. Overall latency [us]

13-1. concept

Latency는 입력을 넣고 run을 준 뒤 최종 output이 완성되어 done이 뜰 때까지 걸리는 시간이다.

계산식) Latency = cycle count x clock period (내 코드에서는 clock period = 10ns)

13-2. 내 hardware에서 어느 부분일까?

Project FSM에서 latency가 결정된다.

핵심 state : S_FC1_RST, S_FC1_RUN, S_FC1_WAIT, S_NORM_RELU, S_FC2_RST, S_FC2_RUN, S_FC2_WAIT, S_NEXT, S_DONE

현재 코드에서는 내부 matmul 완료를 정확한 done으로 받는 대신

localparam [15:0] MATMUL_WAIT = 16'd3000 으로 FC1과 FC2 각각 3000cycle씩 기다린다.

Base의 경우 FC1 wait = 3000 cycles, Norm/ReLU = 64 cycles, FC2 wait = 3000 cycles

Extra의 경우 block_idx=0, block_idx=1 두 번 처리하므로 base 흐름을 2번 반복한다.

1. Base 8x8 latency

FSM overhead는 6~10 cycles이기 때문에 Base latency = 6064 cycles이다.

시간으로는 6064 cycles x 10ns = 60640ns = **60.64us**이다. 실제 console에서 simulation finish time이 약 61.7us 근처로 나왔다.

2. Extra 16x8 latency

Extra는 8x8 block 두 개를 순차 처리하므로

-> extra latency $\approx 2 \times \text{base latency} \approx 2 \times 6064 \text{ cycles} = 12128 \text{ cycles}$

시간은 $12128 \times 10 \text{ ns} = 121,280 \text{ ns} = \mathbf{121.28 \text{ us}}$ 이다.

14. Utilization ratio [%]

14-1. concept

Utilization ratio는 hardware가 가진 최대 연산능력 중 실제 useful operation에 얼마나 사용되었는지를 나타낸다.

계산식) Utilization = useful operations / peak available operations during latency $\times 100$

여기서 useful operations는 실제 FC1, FC2에서 필요한 MAC 연산 수를 의미한다.

14-2. 내 hardware에서 어느 부분인가?

Useful operation은 FC1과 FC2 matrix multiplication이다. Norm/ReLU도 연산이긴 하지만, matrix multiplication에 비해 작고, systolic array utilization 분석에서는 보통 FC MAC 연산 중심으로 계산한다.

1. Base

FC1: $8 \times 8 \text{ output} \times 8 \text{ MAC} = 512 \text{ MAC}$

FC2: $8 \times 8 \text{ output} \times 8 \text{ MAC} = 512 \text{ MAC}$

Total = 1024 MAC

-> 1MAC = 2ops으로 보면 $1024 \text{ MAC} \times 2 \text{ ops} = 2048 \text{ ops}$ 이다.

2. Extra

FC1: $16 \times 8 \text{ output} \times 8 \text{ MAC} = 1024 \text{ MAC}$

FC2: $16 \times 8 \text{ output} \times 8 \text{ MAC} = 1024 \text{ MAC}$

Total = 2048 MAC

-> 1MAC = 2ops으로 보면 $2048 \text{ MAC} \times 2 \text{ ops} = 4096 \text{ ops}$

A. Effective peak 0.8 GOPS 기준

Base:

Peak available ops during base latency = $0.8 \times 10^9 \text{ ops/s} \times 60.64 \times 10^{-6} \text{ s} = 48,512 \text{ ops}$

Utilization = $2048 / 48512 \times 100 \approx 4.22 \%$

Extra:

Peak available ops during extra latency = $0.8 \times 10^9 \times 121.28 \times 10^{-6} = 97,024 \text{ ops}$

Utilization = $4096 / 97024 \times 100 \approx 4.22 \%$

B. Nominal peak 3.2 GOPS 기준

Base:

$3.2 \times 10^9 \times 60.64 \times 10^{-6} = 194,048 \text{ ops}$

Utilization = $2048 / 194048 \times 100 \approx 1.06 \%$

Extra:

$3.2 \times 10^9 \times 121.28 \times 10^{-6} = 388,096 \text{ ops}$

Utilization = $4096 / 388096 \times 100 \approx 1.06 \%$

15. Bandwidth가 2x/0.5x가 되면 latency와 utilization은 어떻게 변하는가?

15-1. concept

memory bandwidth가 커지면 데이터를 더 빨리 읽고 쓸 수 있음

→ memory load/store 시간이 줄어듦

→ latency 감소 가능

→ PE idle time 감소

→ utilization 증가 가능

* 반대로 bandwidth가 작아지면 위와 반대로 된다.

15-2. 내 hardware에서 어느 부분인가?

내 hardware에서는 WHT_LOAD, ACT_LOAD, WRITE가 bandwidth 영향을 받는다. 이 state들은 ctrl 내부 FSM에서 tile 단위로 memory를 읽고 쓰는 부분이다.

FC1에서는 X memory + W1T memory, FC2에서는 X3 memory + W2T memory 그리고 최종 Y write가 있다.

1. bandwidth 2x

현재보다 memory bandwidth가 2배가 되면

- WHT_LOAD 시간이 감소

- ACT_LOAD 시간이 감소
- WRITE 시간이 감소
- tile 준비 시간이 줄어듦
- systolic array가 data를 기다리는 시간이 감소
- overall latency 감소
- utilization 증가

하지만 지금 project는 MATMUL_WAIT = 3000으로 고정된 wait가 크다. 그래서 bandwidth가 2배가 되어도 latency가 절반으로 줄지는 않는다.

2. bandwidth 0.5x

memory bandwidth가 절반이면

- tile load 시간이 증가
- output write 시간이 증가
- FC1/FC2 전체 latency 증가
- PE idle time 증가
- utilization 감소

16. Systolic array size가 2x / 0.5x가 되면 latency와 utilization은 어떻게 변하는가?

16-1. concept

PE 개수가 바뀌면 과연 성능이 어떻게 변할것인가?

Systolic array size가 커지면:

더 많은 PE가 병렬로 계산

- tile 수 감소 가능
- latency 감소
- peak GOPS 증가

작아지면:

PE 수 감소

- 더 많은 tile 반복 필요
- latency 증가
- peak GOPS 감소

16-2. 내 hardware에서 어느 부분인가?

지금 systolic array는 sa8_4x4, 즉 4x4 PE array이다. 그리고 top 안에서 sa8_4x4 U_sa(..)로 인스턴스화한다. 그리고 ctrl은 8x8 matrix multiplication을 4x4 tile 단위로 처리한다. 현재 8x8 matrix는 4x4 tile 4개로 나뉜다. 그리고 hidden dim도 8이라서 hidden tile도 2개다.

1. PE 개수가 2배가 되는 경우

예를 들어 16 PE에서 32 PE가 되면:

- 동시에 처리 가능한 MAC 수 증가
- peak GOPS 약 2배 증가

- 같은 연산량을 더 짧은 시간에 처리 가능
- latency 감소

하지만 matrix size가 작고 memory bandwidth가 충분하지 않으면 모든 PE를 항상 채우기 어렵다.

- utilization은 반드시 2배 증가하지 않음
- data 공급이 부족하면 utilization은 오히려 낮아질 수도 있음

2. 4x4에서 8x8 array가 되는 경우

8x8 SA를 사용하면 8x8 output tile을 한번에 처리할 수 있다. 현재는 8x8 output을 4개의 4x4 tile로 나눈다.

-> 따라서:

- tile 반복 횟수 감소
- weight prefill 횟수 감소
- buffer load/write overhead 감소
- latency 크게 감소
- peak performance 증가

17. Clock speed를 바꾸지 않고 latency를 개선하는 방법

17-1. concept

Frequency를 올리지 않고, architecture나 scheduling 개선만으로 지연시간을 줄이는 방법을 알아보는 것이다.

17-2. 내 hardware에서 어느 부분인가?

내 설계에서 latency를 크게 만드는 부분:

1. MATMUL_WAIT = 3000
2. FC1과 FC2를 같은 matmul engine으로 순차 처리
3. Norm/ReLU 동안 systolic array idle
4. tile load, weight prefill, writeback이 computation과 overlap 안 됨

특히 가장 큰 건 MATMUL_WAIT이다. 이건 안전하게 기다리기 위한 고정 wait cycle이라서 실제 필요한 cycle보다 클 가능성이 크다. project FSM에서 S_FC1_WAIT, S_FC2_WAIT가 wait_cnt < MATMUL_WAIT 조건으로 유지됨. 즉 matmul이 실제로 끝났는지를 보고 넘어가는 게 아니라 일정 cycle을 무조건 기다린다. 또 Norm/ReLU는 S_NORM_RELU에서 element-wise로 수행되고, 이동안 matmul engine은 사용되지 않는다.

17-3. latency를 줄이는 방법

1. fixed wait 제거

Matmul engine에 done signal 추가하면 실제 8x8 matmul이 끝나는 순간 바로 다음 state로 이동한다. 불필요한 wait cycle 제거할 수 있고, latency가 감소한다.

2. double buffering

현재는 tile load -> compute -> write 순서로 진행되는데 현재 tile 계산 중 다음 tile을 미리 load하면 memory load time과 computation time이 overlap되고 latency가 감소한다.

3. Norm/ReLU와 FC2 준비 overlap

현재는 FC1을 전체 완료하고 Norm/ReLU를 진행한 후 FC2를 시작한다. 이것을 FC1 output tile이 나올 때마다 바로 Norm/ReLU를 수행하도록 해주고 그 결과를 X3 buffer에 저장한다. 그 다음 FC2 tile 준비와 overlap한다. 그러면 layer 사이 idle cycle이 감소한다.

18. Utilization ratio를 개선하는 방법

18-1. concept

Utilization 개선은 PE가 노는 시간을 줄이는 것이다. 즉, Systolic array가 가능한 한 많은 cycle 동안 유효한 MAC을 수행하도록 만드는 것이다. 현재 utilization이 낮은 이유는 memory load 중 PE idle, weight prefill 중 일부 PE idle 등의 이유가 있다.

18-2. 내 hardware에서 어느 부분인가?

내 hardware에서 utilization을 결정하는 핵심은 sa8_4x4 U_sa이다. 이 PE들이 실제로 계산하는 시간 대비 전체 latency가 얼마나 긴지가 utilization이다. Waveform을 보면 state가 WAIT, WHT_LOAD, ACT_LOAD, WRITE인데 sa_busy = 0이면 PE가 놓고 있는 구간이다.

18-3. 개선방안

1. fixed wait 줄이기

MATMUL_WAIT = 3000을 실제 done 기반으로 교체하면 불필요한 idle cycle 감소하고 utilization 증가한다.

2. double buffering

현재 tile 계산 중 다음 tile A/B를 미리 load하면 SA가 memory load때문에 기다리는 시간이 감소한다.

3. Norm/ReLU pipelining

현재 Norm/ReLU 동안 SA는 idle이다. 이때 FC1 output이 나오는 즉시 Norm/ReLU 수행하고 동시에 다음 tile 준비 또는 FC2 weight prefill을 수행하면 SA idle time이 감소한다.